EidosSQL

εἶδος · the form, the thing seen

Master Guide — every feature, and how each one is built

An interactive SQL visualizer for DSO 435 · Data Base Management Systems

Version 1.1 · July 2026

Verified against PostgreSQL: 61/61 differential queries identical

> *In ancient Greek, **εἶδος** is the form — the shape by which a thing is known. In modern Greek, a **type** or kind. EidosSQL makes the form of a query visible: one step, one type, one table at a time.*

Contents

Part I — Product tour

1. Layout

A single-page app with a fixed top bar (brand, **Database** picker, **Examples** picker) and two panes:

- **Left pane** — SQL editor, ▶ Visualize button (also ⌘/Ctrl+Enter), error panel, and a collapsible schema reference for the active database.
- **Stage** (right) — the step player: step header, animated table, playback controls, and the step timeline.

Below ~920 px the panes stack vertically (usable on a tablet; desktop is the primary target). The whole UI has light and dark themes following the OS setting.

2. The editor

- **Syntax highlighting** — keywords, strings, numbers, functions, comments, identifiers each get a token color (text-contrast-safe in both themes).
- **Clause spotlight** — while stepping through a visualization, the exact SQL span responsible for the current step is highlighted in blue and auto-scrolled into view. This is the thread that ties "what the query says" to "what the database is doing".
- **Error underline** — parse/runtime errors wavy-underline the exact offending span; the panel below shows the message with a line number and, for classic student mistakes, a 💡 teaching hint (see §12).
- Tab inserts spaces; ⌘/Ctrl+Enter runs. If you edit after running, a "edited — run again to update" note appears and the stale highlight is suppressed.

3. The step player

Pressing **Visualize** replays the query in the database's *logical* clause order — not the order you wrote it:

FROM → JOIN(s) → WHERE → GROUP BY → collapse → HAVING → window functions → SELECT → DISTINCT → set operations → ORDER BY → LIMIT/OFFSET → RESULT

Each step shows:

- **Path badges** — nesting context (`WITH acct_orders`, `UNION — branch 2`, `Subquery in HAVING`), so CTEs, subqueries, and set-op branches read as stories-within-the-story.
- **Title** — the clause, verbatim (`WHERE total > 500`).
- **Description** — plain English with *real row counts* ("14 of 59 rows pass; 45 are removed").

- 💡 **Insight** — an optional teaching callout (why WHERE can't see aliases; what LEFT JOIN promises; what LIMIT without ORDER BY doesn't).
- **The table** — the animated state of the data at that moment.

Controls: ← / → keys, Prev/Next buttons, Play with 0.5×–2× speed, clickable timeline chips (indented `›` for nested steps), and a step counter. Row counts in each description tick up in a quick count-up animation (disabled for reduced-motion users), pulling the eye to the step's payload.

Timeline chips are color-coded by source. Every chip is tinted by the table(s), CTE(s), or subqueries its step is working on — assigned stable colors in order of first appearance and listed in a legend above the timeline. A JOIN chip is shaded **half-and-half** with its two sides' colors; steps inside a CTE wear that CTE's ingredients' colors; the "CTE ready" chip introduces the CTE's own color, which then reappears wherever the main query uses it — including inside a HAVING subquery that reads it. The goal: students can glance at the chip row and know *which table each clause is about* without re-reading the query, precisely when CTEs and subqueries make that hardest. A Greek-key (meander) rule underlines the timeline — one of exactly two Greek identity touches in the UI (the other is the wordmark's column-capital "E").

4. The animated table — reading the visuals

Rows keep a **stable identity** across steps, so the animation between two steps *is* the explanation:

Visual	Meaning
✓ green row	passed this step's test (WHERE/HAVING/LIMIT keep)
× red row, dimmed	failed; it fades out on the next step
+ blue row	appeared at this step (NULL-filled outer-join row, group row)
Colored left edge + wash	group / partition / join-provenance membership (8-color validated palette)
Blue-underlined column	column computed at this step (aggregate, window value, expression)
Gray pill on a row	per-row note ("no match — removed by INNER JOIN", "duplicate removed by UNION")
NULL in italics	a real SQL NULL, styled distinctly from text
Rows sliding	ORDER BY reordering (FLIP animation — no row appears or disappears)

Steps display the first 100 rows (with an "… N more rows" footer); the final RESULT view shows up to 1,000. Computation always covers *all* loaded rows — the caps are presentation only.

5. Databases

- **Embedded: Parch & Posey** — the class teaching database, trimmed to a referentially-intact 12-account slice (30 orders, 32 web events, all 7 regions, 10 reps) so every row fits on screen. The trim deliberately keeps teaching edge cases: **Mattel** has no orders (LEFT JOIN lessons), **International** has a rep but no accounts, **South/North** have nothing.
- **Connect your own Postgres** (Database → "➕ Connect your own Postgres…") — point the app at any database on your machine (`postgres://localhost:5432/northwind` , or just `northwind`). Choose a sample size from 100 rows up to **all rows** (50,000/table hard cap). If any table is sampled, the schema panel labels it ("300 of 6,912 rows (sample)") and a persistent amber banner warns that results on a sample can differ from the full database. With "all rows", results are exact.
- **Load CSV files** (Database → "📄 Load CSV files…") — each file becomes a table named after the file; the first row supplies column names and types are inferred per column (integer, numeric, date, timestamp, boolean, text; blank cells become NULL). Everything parses in the browser — nothing is uploaded anywhere. This lets an instructor hand out any dataset as plain CSVs with zero database setup.

5a. Presentation mode, themes, and printing

- 🎬 **Present** (top bar, Esc to exit) is the projector view: the editor pane disappears, a read-only syntax-highlighted copy of the query docks above the stage — keeping the clause spotlight visible — and type sizes step up across the board.
- ☀️ /🌙 **theme toggle** — light and dark are both first-class; the manual choice persists and overrides the OS setting (projectors want light mode regardless of the presenter's laptop).
- **Printing** any step produces a clean handout: light-forced colors, the full query with highlighting, the step narration, and the table laid out for paper with the app chrome stripped — homework material for free.

6. Examples

~20 curated queries in the Examples menu, ordered like the semester:

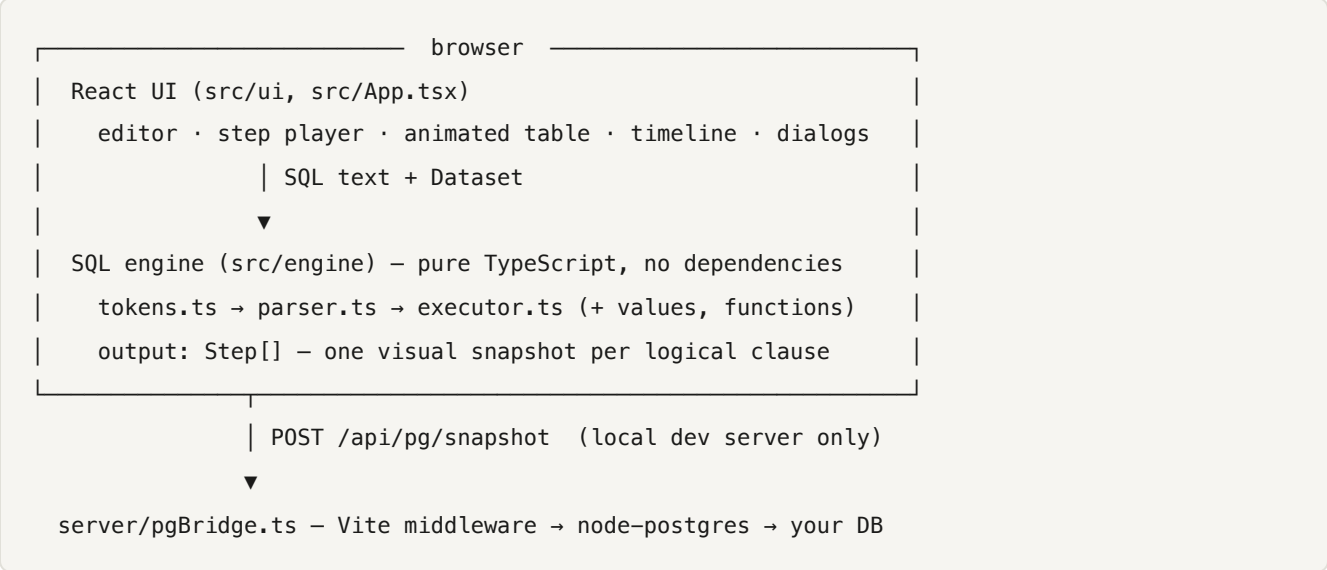
1. **Foundations** — SELECT, WHERE, ORDER BY + LIMIT, CASE
2. **Joins** — JOIN, chained joins, LEFT JOIN, LEFT JOIN + IS NULL
3. **Aggregation** — GROUP BY, HAVING
4. **Subqueries & CTEs** — IN, scalar subquery, CTE, UNION
5. **Window functions** — running total, RANK per region, LAG, NTILE

6. **Putting it together** — three capstones at end-of-course difficulty: UNION + CTE + CASE · LEFT JOIN + HAVING-with-subquery · LAG inside a CTE

Selecting an example loads the query, switches to the right database, and runs it immediately.

Part II — How it's built

7. Architecture at a glance



The engine is written from scratch (no sql.js, no parser library) for one reason: **off-the-shelf engines only give you the final answer**. To animate intermediate states, the evaluator itself must pause after every logical clause, snapshot the working relation, and keep row identities stable. That requirement shaped everything below.

File map:

```
src/engine/tokens.ts      tokenizer + SqlError (positions, hints)
src/engine/ast.ts         AST node types; every node carries a source span
src/engine/parser.ts      recursive-descent parser
src/engine/values.ts      value semantics: NULL logic, dates, intervals
src/engine/functions.ts   scalar + aggregate function library
src/engine/executor.ts    the evaluator/step-emitter (the heart, ~1,400 loc)
src/engine/steps.ts       Step/VizTable model consumed by the UI
src/data/datasets.ts      embedded Parch & Posey slice (generated from live DB)
src/data/remote.ts        client for the Postgres bridge
src/data/csv.ts           CSV parser + type inference → Dataset
src/ui/*                  Editor, StepTable, Timeline (source-colored chips),
                          SchemaPanel, ConnectPanel, CsvPanel, SqlView
                          (read-only SQL for present/print), AnimatedDesc
server/pgBridge.ts        local-Postgres bridge (Vite dev/preview middleware)
scripts/smoke.ts          engine smoke test (26 cases, incl. every example)
scripts/verify.ts         engine-vs-Postgres differential test (61 cases)
```

8. Value semantics (`values.ts`)

- Values are plain JS: `number | string | boolean | null`, plus an `Interval` object for timestamp arithmetic.
- **Dates/timestamps are ISO strings** (`2016-12-24 05:53:13`). Deliberate: they display exactly as Postgres prints them, compare correctly as plain strings, and never drift through timezones. Date math parses them as UTC on demand.
- **Three-valued logic** is enforced at the comparison layer: `compareValues` returns `null` when either side is NULL; `truthy()` implements "NULL and false both reject the row"; AND/OR/NOT propagate unknowns Postgres-style. This is why `NULL = NULL` correctly filters rows out and `x IN (…, NULL)` returns NULL rather than false.
- **Interval decomposition** mirrors Postgres: a timestamp difference is days + hh:mm:ss, so `EXTRACT(hour FROM (a - b))` gives the *hour component*, not total hours — matching real query results.
- `groupKey(values)` builds the canonical string key used everywhere equality-with-NULLs matters (GROUP BY, DISTINCT, set ops, hash joins, IN sets) — with SQL's twist that NULLs *are* equal for grouping.
- **Integer division**: Postgres computes `7 / 2 = 3`. The engine reproduces this via lightweight static typing — dataset columns carry an `isInt` flag, and `isIntExpr()` propagates int-ness through

literals, arithmetic, `count/sum/min/max`, CASE, and casts (memoized per node). Division truncates only when both operand types are integers. A classic gotcha the course teaches, faithfully reproduced.

9. Tokenizer and parser

`tokens.ts` produces tokens with byte-exact source positions (words, numbers, strings with `''` escapes, quoted identifiers, operators, both comment styles). The same tokenizer drives editor syntax highlighting, so the editor and engine can never disagree about what a token is.

`parser.ts` is a hand-written recursive-descent parser. Coverage:

- SELECT [DISTINCT] with expressions, aliases (`AS` or bare), `*` / `t.*`
- FROM with table aliases, subqueries-as-tables, and every join type (INNER/LEFT/RIGHT/FULL [OUTER]/CROSS, comma joins, ON and USING)
- WHERE · GROUP BY (expressions, ordinals, select aliases) · HAVING
- ORDER BY (expressions, ordinals, output names, ASC/DESC, NULLS FIRST/LAST) · LIMIT/OFFSET
- WITH (multiple CTEs, visible to later CTEs and the body)
- UNION [ALL] / INTERSECT / EXCEPT with correct precedence (INTERSECT binds tighter) and parenthesized branches that may contain their own WITH/ORDER
- Full expression grammar: OR→AND→NOT→comparisons (=, <>, <, <=, >, >=, IS [NOT] NULL/TRUE/FALSE, [NOT] IN list/subquery, BETWEEN, LIKE/ILIKE) →additive (+ − ||)→multiplicative→unary→ `::type` casts→primary
- CASE (searched and operand forms), CAST(x AS type), EXTRACT(field FROM x), INTERVAL '…', DATE/TIMESTAMP literals, EXISTS(…), scalar subqueries, function calls with DISTINCT args and OVER (PARTITION BY … ORDER BY … [ROWS frames])

Two design choices matter most:

1. **Every AST node records `{start, end}`** — that's what powers the editor's clause spotlight, error underlines, and each step's `span`.
2. **Errors are written for students.** The parser knows the difference between "syntax error" and "the mistake a student makes": a LEFT JOIN missing its ON explains cross-join blowup; a FROM-subquery without an alias explains *why* SQL needs the name; keywords used as names suggest double quotes.

10. The executor — evaluation as storytelling

`executor.ts` evaluates the query for real while recording a `Step` after every logical stage. The `Step` model (`steps.ts`):

```
{ phase, chip, title, desc, insight?, span?, path[],   // narration
  table: { columns[], rows[], truncated? } }           // the visual state
```

Stable row identity is the core trick. Base rows are `alias:table:i`; a joined row is `left⋈right`; a NULL-extended row is `left⋈ø`; a group row is `g:<key>`; set-op branches are prefixed `a:` / `b:`. Because ids persist across steps, the UI's FLIP layout animation renders "this row moved / survived / vanished" without the engine knowing anything about animation.

Source tracking works the same way for tables: every relation carries `sources` — the display names of the base tables, CTEs, and subquery aliases it derives from. FROM sets it, JOIN concatenates both sides, grouping/projection/set-ops carry or merge it, and each emitted step tags its sources with a stable color slot (assigned in order of first appearance). The timeline renders those tags as chip tints — one wash, a half-and-half join split, or stripes — plus the legend.

Pipeline details, in order:

- **FROM** resolves against the environment (dataset tables + CTEs), instantiates fresh column ids with the alias as `source` (that's the small gray label over each column), and errors on duplicate aliases with a self-join hint. Subqueries in FROM run first as a nested step group.
- **JOIN** — see §11.
- **WHERE** evaluates per row (with aggregate/window usage rejected with hints), snapshots every row as kept/dropped, then keeps the survivors.
- **GROUP BY** resolves keys three ways (expression, ordinal, select alias — matching Postgres), builds groups in order of first appearance, and emits *two* steps: rows recolored & sorted into contiguous groups, then the collapse into one row per group. The grouped relation's columns are the keys plus every aggregate the rest of the query needs — collected up front by scanning SELECT, HAVING, and ORDER BY (deduped by normalized text; labeled by alias when a select item matches). Aggregates over an empty input with no GROUP BY still produce one row (`count(*)` = 0), a real SQL quirk the differential test caught.
- **Group-context evaluation**: after collapse, expressions are evaluated against a group context where (a) anything textually matching a GROUP BY key returns the key value, (b) a column reference resolving to a key column works under any alias spelling, (c) aggregate calls compute (and cache) over the group's member rows, and (d) any other bare column throws the canonical teaching error ("must appear in GROUP BY or be used in an aggregate — SQL no longer knows *which* row's value to show").

- **HAVING** is WHERE for group rows — same kept/dropped visual, plus the insight that explains the WHERE/HAVING division of labor.
- **Window functions** run after HAVING, over whatever rows remain (including grouped rows — `rank() over (order by sum(x))` works). Implementation: partition by `PARTITION BY` values → sort within partitions (stable, NULLS-aware) → compute per function. Supported: `row_number, rank, dense_rank, ntile, lag, lead, first_value, last_value` and any aggregate with OVER. Frames follow Postgres defaults — with an ORDER BY, the frame is "start through current row *including peers*", which is why running totals tie on equal keys — plus explicit `ROWS BETWEEN n PRECEDING AND CURRENT ROW` frames for moving averages. The step shows partitions striped in color and the computed columns appearing; afterwards the relation keeps the window order (like Postgres output, and it makes LAG values visually traceable).
- **SELECT** happens *late*, as in real SQL: stars expand (excluding internal window columns), carried columns keep their ids (so the UI can show untouched columns persisting), computed columns get fresh ids and a "new" highlight. Labels: alias > column name > shortened expression text.
- **DISTINCT** dedupes whole output rows via `groupKey`, marking duplicates before removing them.
- **Set operations** run each branch as its own step group, enforce equal column counts (with a teaching message), stack rows with branch coloring, and show duplicate elimination (UNION), membership testing (INTERSECT), or subtraction (EXCEPT) row by row.
- **ORDER BY** resolves each key as output ordinal → unique output name → arbitrary expression evaluated against the *pre-projection* row (kept alongside by row id) — mirroring Postgres's ability to sort by non-selected columns, including its restriction that with DISTINCT the sort keys must be in the select list (enforced, with the explanation). Sorting is stable, multi-key, DESC-aware, with Postgres NULL defaults.
- **LIMIT/OFFSET** marks the cut rows before dropping them, and warns when there's no ORDER BY ("LIMIT keeps an *arbitrary* N rows").
- **RESULT** — the clean final table.

11. Joins: planner, hash path, and honesty about non-matches

The ON expression is flattened into AND-conjuncts. Each `=` conjunct is classified by which side its columns come from; clean left/right splits become **equi-join keys**, everything else stays a per-pair *residual* test.

- **Hash path** (any equi-keys): build a hash map of right-side rows keyed by `groupKey(keys)` (rows with NULL keys are excluded — NULL never equals anything), then probe from each left row and apply residuals to the candidates. Bucket insertion order preserves table order, so output is identical to the nested-loop result. This is what makes full-size tables (7,000 × 9,000 rows) join in milliseconds.

- **Nested-loop path** (no equality at all — inequality joins, cross joins): every pair is tested, guarded at 1.5M pairs with a hint to add an ON equality or pre-filter.
- **Outer-join semantics**: unmatched left rows appear as NULL-filled "+" rows (LEFT/FULL) or as × ghost rows annotated "no match — removed by INNER JOIN" (the single most clarifying visual in the app: inner vs. outer join is *shown*, not told). RIGHT/FULL append unmatched right rows. Matched rows are tinted by left-row provenance so students can see one account "fanning out" across its orders.
- `USING (col)` is rewritten to explicit equalities. (Deviation: Postgres merges the USING column into one; EidosSQL keeps both sides' columns.)
- Guards: 500k result rows max; a global **work meter** (30M evaluation ticks) aborts any runaway query with a teaching hint instead of freezing the tab.

12. Subqueries, CTEs, and correlation

- **CTEs** execute first as nested step groups, then register as tables in a scoped environment (visible to later CTEs and the body — including inside subqueries, which is how `(select avg(n) from some_cte)` works).
- **Scalar / IN / EXISTS subqueries** evaluate lazily at expression time. Uncorrelated ones are cached after one evaluation and — key pedagogy — their steps are emitted *inline, right before the clause that uses them*, ending with a "Subquery result: 2.125" step so students see the value get plugged in.
- **Correlation detection** needs no static analysis: when a subquery evaluates, the enclosing row context sits on an outer-scope stack with a tripwire flag. If column resolution falls through to the outer scope, the flag trips → the subquery is correlated → it's not cached, it re-runs per row (silently after the first, whose steps are shown as the illustrative case), and the wrap-up step explains exactly that.
- `IN (subquery)` membership uses a hashed set (with SQL's NULL-in-list semantics preserved), so it stays linear on full tables.
- Scalar subqueries enforce Postgres's contract with explanations: one column ("used as a single value"), at most one row ("use IN instead of =" when many rows come back), NULL when empty.

13. The teaching-error catalog

Errors are the app's second curriculum. The notable ones:

Mistake	What EidosSQL says
SELECT alias used in WHERE/HAVING	"WHERE runs *before* SELECT names its outputs… repeat the expression, or filter a subquery/CTE"
Aggregate in WHERE	"WHERE filters rows before grouping — use HAVING"
Bare column with GROUP BY	"SQL no longer knows *which* single value to show — group by it or aggregate it"
Window function in WHERE/HAVING	"computed near the end — compute it in a CTE, then filter"
`"Walmart"` in double quotes	"double quotes name a *column*; use single quotes for text"
LEFT JOIN without ON	"without ON, every row pairs with every row"
FROM-subquery without alias	why the name is required, with the fix
Scalar subquery returns N rows	"comparing against many values? use IN"
Nested aggregates	"compute the inner aggregate in a CTE first"
Ambiguous column	lists the candidate tables, shows the qualified form
Unknown column/table/function	lists what *is* available
Division by zero	suggests `NULLIF(denominator, 0)`
UNION column-count mismatch	"set operations stack results vertically…"
WHERE after GROUP BY	word-order fix with the WHERE/HAVING rule

14. The Postgres bridge (`server/pgBridge.ts`)

Browsers can't speak the Postgres wire protocol, so "connect your own database" is a ~150-line middleware living *inside the student's own Vite dev server* (`configureServer` / `configurePreviewServer`):

- One endpoint: `POST /api/pg/snapshot {conn, limit}` → database name + every public-schema base table (max 40) with columns, total row count, and up to `limit` rows (hard cap 50,000; `limit ≤ 0` means "all").
- The session is forced `READ ONLY`; only catalog queries and `SELECT … LIMIT` are ever issued; identifiers are quote-escaped.
- Type mapping: Postgres `data_type` → the engine's `integer/numeric/text/timestamp/date/boolean`; `pg` type parsers are overridden so numerics arrive as numbers and dates/timestamps as strings, then normalized to the engine's canonical formats (strip `T`, ms, tz).

- Connection strings never persist server-side and never leave the machine; a bare name like `northwind` expands to `postgres://localhost:5432/…`.
- On a static deployment the endpoint doesn't exist; the client detects the 404/network failure and explains that this feature needs `npm run dev`.

15. UI implementation notes

- **Editor** = a transparent `<textarea>` stacked on a highlighted `<pre>` with synced scroll — the classic overlay technique, but the highlight layer is built from the *engine's own tokenizer*, with span-splitting to overlay the active-clause and error ranges.
- **StepTable** renders `motion.tr` rows keyed by stable row id inside `AnimatePresence`: exits fade, survivors FLIP into place, entries fade in. Above 150 rows (the 1,000-row result view) it switches to static `<tr>`s — animating a thousand rows would burn CPU for zero pedagogy. Numeric cells right-align with `tabular-nums`.
- **Timeline** chips auto-scroll the active chip into view; nesting depth renders as `›` prefixes and dashed borders.
- **App state** is deliberately boring React `useState`: the run is an immutable `Step[]`, so Prev/Next/Play/scrub are just an index change — stepping backwards is free.
- Playback pace is 2.4 s/step ÷ speed; keyboard arrows are ignored while typing in inputs; `MotionConfig reducedMotion="user"` plus a CSS reduced-motion block respect accessibility settings.
- **Presentation mode** is one boolean and a CSS class: `.presenting` hides the editor pane, reveals the docked `SqlView` (the editor's own segment-builder rendering into a read-only `<pre>`, spotlight included), and scales typography. Esc exits. **Print** reuses the same `SqlView` via `@media print`, which also forces light tokens and unclips the table.
- **Theme** is a `data-theme` attribute on `<html>`: dark tokens apply under `prefers-color-scheme: dark` *unless* the user chose light, and under an explicit dark choice regardless of the OS. The choice persists in localStorage.
- **Count-up descriptions** split the text on number tokens and animate each integer 0→value over ~500 ms (cubic ease-out, `requestAnimationFrame`), snapping to the exact original formatting at the end; decimals and reduced-motion users render statically.
- **CSV ingestion** (`src/data/csv.ts`) is a hand-rolled RFC-4180 parser (quoted fields, `""` escapes, CRLF) plus per-column type inference by regex consensus over non-empty values; conversion mirrors the embedded dataset's value conventions so the engine treats all three sources identically.

16. Design system

Tokens follow a validated reference palette (categorical slots ordered to maximize worst-case color-vision-deficiency separation — verified with a palette validator in both light and dark, against each theme's actual surface):

- **Status colors** (kept/dropped/new) are semantically reserved and always paired with a glyph (✓ × +) and often a note — color is never the only carrier.
- **Group/partition colors** are 10%-opacity washes plus a 3px left edge; the actual key values are always visible as text in the row.
- Chrome follows the token sheet (`--page/--surface/--ink/--grid/…`) with a full dark variant; code tokens are darkened/lightened per theme for text-grade contrast.
- System font stack; monospace reserved for SQL, identifiers, and the step title (which quotes the query).

17. Datasets

`src/data/datasets.ts` is generated from the live class database: 12 well-known accounts chosen to span four regions and nine reps, each with its first/middle/last orders and web events so time-series examples work, plus intact edge cases (§5). Rows are stored as compact JSON arrays with typed column definitions — the `integer` markers are what power Postgres-style integer division (§8).

Part III — Trust, limits, and workflows

18. Verification

Two layers, both runnable any time:

- `npm run smoke` — 26 engine cases (foundations through capstones, including **every curated example**) printing step traces and results.
- `npm run verify` — the **differential test**: creates a scratch local Postgres database (`eidossql_verify`), loads the exact embedded dataset, runs a 61-query battery through both the engine and Postgres, and diffs results cell-by-cell (NULL-tokenized, float-tolerant, multiset comparison unless the query has a determining ORDER BY), then drops the database. Coverage: every join type on both planner paths, grouping edge cases, all window function classes and frames, set ops, correlated and uncorrelated subqueries, date/interval math, casts, integer division, and the three capstones. **Current status: 61/61 identical.**

The differential harness has already earned its keep — it caught two real engine bugs during development (aggregate-over-zero-rows returning no row instead of one; window-over-grouped-rows failing its value lookup) that no amount of eyeballing would have found.

19. Known limitations & deviations from Postgres

Statements: SELECT only — no INSERT/UPDATE/DELETE/DDL, one statement at a time. Within SELECT, not supported: `ANY` / `ALL` / `SOME` comparisons, `NATURAL JOIN`, `LATERAL`, named windows (`WINDOW w AS …`), `GROUPING SETS/ROLLUP/CUBE`, `FILTER (WHERE …)`, `WITHIN GROUP` /ordered-set aggregates, `string_agg` / `array_agg`, arrays/JSON operators, `RANGE` / `GROUPS` explicit frames (`ROWS` frames and defaults are supported), recursive CTEs, `TABLESAMPLE`, collation controls.

Deliberate deviations: `USING` keeps both join columns instead of merging (§11); `to_char` implements the common patterns only; very lenient number-vs-numeric-string comparisons (teaching kindness over strictness); row output order without ORDER BY follows the engine's deterministic pipeline order (Postgres promises nothing there anyway).

Practical ceilings: 50,000 rows/table from the bridge, 500k join-result rows, 30M evaluation ticks, 100 rows displayed per step / 1,000 in the result view.

20. Workflows

```
npm run dev      # dev server + Postgres bridge  → http://localhost:5173
npm run build    # static production build (dist/) — bridge not included
npm run preview  # serve the build locally (bridge included)
npm run smoke    # engine smoke test
npm run verify   # differential test vs local Postgres (needs psql/createdb)
```

Environment note: if `npm run dev` fails with "Cannot find native binding" (a known npm optional-dependency bug), run `npm install --no-save @rolldown/binding-darwin-arm64@<rolldown version>`.